

Python Interview Questions

Beginner

1. If you have data with names of products and their prices, which Python data type will you use to store it and why?

2. What are the main benefits of Python?

3. What is the output of the following code in Python?

```
>>>name = 'Codebasics'
>>>print(name[:4] + name[4:])
```

4. You have a list of top 5 MAANG companies and want to print each company with its position in the list.

What is the easiest way to print the position and the company name (like “0 Meta”, “1 Amazon”, etc.) without using a separate counter variable?

5. You ask the user to enter their age using input(), but sometimes they type words like “twenty” instead of numbers.

How will you handle this error in Python so that the program shows a friendly message like “Please enter a valid number!” instead of crashing?

6. In Python, you have two lists. One contains existing data, and the other contains new items to be added.

How will you decide whether to use append() or extend()? Also, explain the difference between the two with a suitable example.

7. What is a Module in Python?

8. What is the use of // operator in Python?

9. What is the use of the zip() function in Python? Explain with an example.

10. You’re building a library app. You need to represent a book with its title and author. Create a class Book and an object representing “Harry Potter” by “J.K. Rowling”.

11. What is the role of the `__init__()` method in Python classes? How does the `self` keyword work, and why is it important in defining instance methods and variables?
12. Explain how file handling is done in Python. Describe the different file modes available and how they affect file operations.

Intermediate

1. How will you specify source code encoding in a Python source file?
2. What is the use of PEP 8 in Python?
3. How does memory management work in Python?
4. How can you retrieve data from a MySQL database using a Python script? Explain the steps involved and mention the functions used for querying and fetching data.
5. Is Python an Object-Oriented or Functional Programming Language?
6. What are Magic Functions in Python?
7. What is a Lambda Expression in Python?
8. What is inheritance in Python and why is it useful? Explain how method overriding works in the context of inheritance. Create a base class `Shape` and a subclass `Rectangle` that overrides the `area()` method.
9. What is the purpose of the `super()` function in Python?
10. Explain how string immutability in Python affects string operations. How does Python handle operations like concatenation, slicing, or replacement if strings cannot be changed? Discuss the advantages and disadvantages of string immutability with examples.

Advanced

1. You're building a student ranking system.
 - a. You have a list of student names along with their scores, like this:

```
students = [ ("Alice", 88), ("Bob", 75), ("Charlie", 95), ("David", 82) ]
```

- b. How can you sort this list based on the students' scores in descending order using a lambda expression?
2. What is a Metaclass in Python?
3. What is the Global Interpreter Lock (GIL) in Python, and how does it impact multithreading?
4. What are the different debugging techniques used in Python?
5. You're working on a large-scale Python project split across multiple teams. The project has the following structure:

```
my_project/
|
├─ auth/
|   ├── __init__.py
|   ├── login.py
|   └── register.py
|
├─ billing/
|   ├── __init__.py
|   ├── invoice.py
|   └── payment.py
|
├─ utils/
|   ├── __init__.py
|   └── validators.py
|
├─ main.py
└─ requirements.txt
```

How would you convert this into a distributable Python package?

6. You are given a nested list of employee records, where each record is a tuple containing (name, department, salary).

```
employees = [ ("Alice", "Tech", 80000), ("Bob", "HR", 50000), ("Charlie", "Tech", 72000), ("Diana", "Tech", 88000), ("Eve", "Sales", 60000), ]
```

You are tasked to:

- a. Extract the names of employees in the "Tech" department who earn more than 75,000.

- b. Convert all such names to uppercase.
 - c. Implement using a single list comprehension.
 - d. Additionally, explain the trade-offs between using list comprehensions and traditional for-loops.
7. In a large enterprise application, you are asked to implement the following features:
- a. Automatically log access whenever certain functions are called. (e.g., `update_profile`, `process_payment`)
 - b. Add retry logic to functions that may intermittently fail. (e.g., network calls)
 - c. Apply the same logging and retry logic without modifying the original function code. How would Python decorators help in solving this?

What are the advantages of using decorators for such functionality over traditional function wrappers or class inheritance? Also explain how decorators can be stacked.

8. You are building a data pipeline that processes large batches of records from a database. How can Python generators help you process this data efficiently? Demonstrate with an example.
9. You are designing a payment processing system that supports multiple payment methods. How would you use abstract classes in Python to enforce a consistent interface across payment methods? What are the benefits of using abstraction?
10. What are the SOLID Principles?